VELORA — Module Tenant (Part 2)

DTOs · Requests · Resources · Services · Controllers · Routes · Middleware

7. DTOs

app/Modules/Tenant/DTOs/CreateBusinessDTO.php

```php
<?php

namespace App\Modules\Tenant\DTOs;

class CreateBusinessDTO
{
    public function __construct(
        public readonly string  $name,
        public readonly string  $email,
        public readonly string  $ownerId,
        public readonly ?string $phone    = null,
        public readonly ?string $address  = null,
        public readonly ?string $city     = null,
        public readonly string  $currency = 'IDR',
        public readonly string  $timezone = 'Asia/Jakarta',
        public readonly string  $country  = 'ID',
    ) {}

    public static function fromRequest(array $data): static
    {
        return new static(
            name:     $data['name'],
            email:    $data['email'],
            ownerId:  $data['owner_id'],
            phone:    $data['phone']    ?? null,
            address:  $data['address'] ?? null,
            city:     $data['city']    ?? null,
            currency: $data['currency'] ?? 'IDR',
            timezone: $data['timezone'] ?? 'Asia/Jakarta',
            country:  $data['country']  ?? 'ID',
        );
    }

    public function toArray(): array
    {
        return [
            'name'     => $this->name,
            'email'    => $this->email,
```

```
            'owner_id' => $this->ownerId,
            'phone'    => $this->phone,
            'address'  => $this->address,
            'city'     => $this->city,
            'currency' => $this->currency,
            'timezone' => $this->timezone,
            'country'  => $this->country,
        ];
    }
}
```

app/Modules/Tenant/DTOs/UpdateBusinessDTO.php

```php
<?php

namespace App\Modules\Tenant\DTOs;

class UpdateBusinessDTO
{
    public function __construct(
        public readonly ?string $name       = null,
        public readonly ?string $phone      = null,
        public readonly ?string $address    = null,
        public readonly ?string $city       = null,
        public readonly ?string $province   = null,
        public readonly ?string $postalCode = null,
        public readonly ?string $taxId      = null,
        public readonly ?string $language   = null,
    ) {}

    public static function fromRequest(array $data): static
    {
        return new static(
            name:       $data['name']        ?? null,
            phone:      $data['phone']       ?? null,
            address:    $data['address']     ?? null,
            city:       $data['city']        ?? null,
            province:   $data['province']    ?? null,
            postalCode: $data['postal_code'] ?? null,
            taxId:      $data['tax_id']      ?? null,
            language:   $data['language']    ?? null,
        );
    }

    public function toArray(): array
    {
        return array_filter([
            'name'        => $this->name,
            'phone'       => $this->phone,
            'address'     => $this->address,
            'city'        => $this->city,
            'province'    => $this->province,
            'postal_code' => $this->postalCode,
            'tax_id'      => $this->taxId,
```

```
            'language'    => $this->language,
        ], fn($v) => $v !== null);
    }
}
```

app/Modules/Tenant/DTOs/CreateBranchDTO.php

```php
<?php

namespace App\Modules\Tenant\DTOs;

class CreateBranchDTO
{
    public function __construct(
        public readonly string  $businessId,
        public readonly string  $name,
        public readonly ?string $code    = null,
        public readonly ?string $address = null,
        public readonly ?string $city    = null,
        public readonly ?string $phone   = null,
        public readonly ?string $email   = null,
        public readonly bool    $isMain  = false,
    ) {}

    public static function fromRequest(array $data, string $businessId): static
    {
        return new static(
            businessId: $businessId,
            name:       $data['name'],
            code:       $data['code']    ?? null,
            address:    $data['address'] ?? null,
            city:       $data['city']    ?? null,
            phone:      $data['phone']   ?? null,
            email:      $data['email']   ?? null,
            isMain:     $data['is_main'] ?? false,
        );
    }

    public function toArray(): array
    {
        return [
            'business_id' => $this->businessId,
            'name'        => $this->name,
            'code'        => $this->code,
            'address'     => $this->address,
            'city'        => $this->city,
            'phone'       => $this->phone,
            'email'       => $this->email,
```

```
            'is_main'     => $this->isMain,
        ];
    }
}
```

8. Form Requests

app/Modules/Tenant/Requests/StoreBusinessRequest.php

```php
<?php

namespace App\Modules\Tenant\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreBusinessRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'     => ['required', 'string', 'max:150'],
            'email'    => ['required', 'email', 'unique:businesses,email'],
            'owner_id' => ['required', 'uuid', 'exists:users,id'],
            'phone'    => ['nullable', 'string', 'max:20'],
            'address'  => ['nullable', 'string'],
            'city'     => ['nullable', 'string', 'max:100'],
            'currency' => ['nullable', 'string', 'size:3'],
            'timezone' => ['nullable', 'string', 'timezone:all'],
            'country'  => ['nullable', 'string', 'size:2'],
        ];
    }
}
```

app/Modules/Tenant/Requests/UpdateBusinessRequest.php

```php
<?php

namespace App\Modules\Tenant\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateBusinessRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'        => ['sometimes', 'string', 'max:150'],
            'phone'       => ['sometimes', 'string', 'max:20'],
            'address'     => ['sometimes', 'string'],
            'city'        => ['sometimes', 'string', 'max:100'],
            'province'    => ['sometimes', 'string', 'max:100'],
            'postal_code' => ['sometimes', 'string', 'max:10'],
            'tax_id'      => ['sometimes', 'string', 'max:30'],
            'language'    => ['sometimes', 'string', 'in:id,en'],
        ];
    }
}
```

app/Modules/Tenant/Requests/StoreBranchRequest.php

```php
<?php

namespace App\Modules\Tenant\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class StoreBranchRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        $businessId = $this->route('business') ?? auth()->user()->business_id;

        return [
            'name'    => ['required', 'string', 'max:100'],
            'code'    => [
                'nullable', 'string', 'max:20',
                Rule::unique('branches')->where('business_id', $businessId),
            ],
            'address' => ['nullable', 'string'],
            'city'    => ['nullable', 'string', 'max:100'],
            'phone'   => ['nullable', 'string', 'max:20'],
            'email'   => ['nullable', 'email', 'max:150'],
            'is_main' => ['nullable', 'boolean'],
        ];
    }
}
```

app/Modules/Tenant/Requests/UpdateBranchRequest.php

```php
<?php

namespace App\Modules\Tenant\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateBranchRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'    => ['sometimes', 'string', 'max:100'],
            'address' => ['sometimes', 'string'],
            'city'    => ['sometimes', 'string', 'max:100'],
            'phone'   => ['sometimes', 'string', 'max:20'],
            'email'   => ['sometimes', 'email', 'max:150'],
            'status'  => ['sometimes', 'string', 'in:active,inactive,closed'],
        ];
    }
}
```

9. Resources

app/Modules/Tenant/Resources/BusinessResource.php

```php
<?php

namespace App\Modules\Tenant\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class BusinessResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'          => $this->id,
            'name'        => $this->name,
            'slug'        => $this->slug,
            'email'       => $this->email,
            'phone'       => $this->phone,
            'address'     => $this->address,
            'city'        => $this->city,
            'province'    => $this->province,
            'country'     => $this->country,
            'currency'    => $this->currency,
            'timezone'    => $this->timezone,
            'language'    => $this->language,
            'tax_id'      => $this->tax_id,
            'logo_url'    => $this->logo_url,
            'status'      => $this->status,
            'status_label'=> $this->status?->label(),
            'verified_at' => $this->verified_at?->toIso8601String(),
            'owner'       => $this->whenLoaded('owner', fn() => [
                'id'    => $this->owner->id,
                'name'  => $this->owner->name,
                'email' => $this->owner->email,
            ]),
            'main_branch' => $this->whenLoaded('mainBranch', fn() =>
                new BranchResource($this->mainBranch)
            ),
            'branches_count' => $this->whenCounted('branches'),
```

```
            'created_at'  => $this->created_at?->toIso8601String(),
            'updated_at'  => $this->updated_at?->toIso8601String(),
        ];
    }
}
```

app/Modules/Tenant/Resources/BranchResource.php

```
<?php

namespace App\Modules\Tenant\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class BranchResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'          => $this->id,
            'business_id' => $this->business_id,
            'name'        => $this->name,
            'code'        => $this->code,
            'address'     => $this->address,
            'city'        => $this->city,
            'phone'       => $this->phone,
            'email'       => $this->email,
            'is_main'     => $this->is_main,
            'status'      => $this->status,
            'status_label'=> $this->status?->label(),
            'created_at'  => $this->created_at?->toIso8601String(),
        ];
    }
}
```

10. Services

app/Modules/Tenant/Services/BusinessService.php

```php
<?php

namespace App\Modules\Tenant\Services;

use App\Modules\Tenant\DTOs\CreateBusinessDTO;
use App\Modules\Tenant\DTOs\UpdateBusinessDTO;
use App\Modules\Tenant\Events\BusinessCreated;
use App\Modules\Tenant\Models\Business;
use App\Modules\Tenant\Repositories\Contracts\BusinessRepositoryInterface;
use App\Shared\Exceptions\VeloraException;
use App\Shared\Services\BaseService;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Storage;

class BusinessService extends BaseService
{
    public function __construct(
        private readonly BusinessRepositoryInterface $repo
    ) {}

    public function getAll(array $filters = []): LengthAwarePaginator
    {
        return $this->repo->all($filters);
    }

    public function findOrFail(string $id): Business
    {
        $business = $this->repo->findById($id);

        if (!$business) {
            throw VeloraException::notFound('Business');
        }

        return $business;
    }

    public function create(CreateBusinessDTO $dto): Business
```

```
    {
        return DB::transaction(function () use ($dto) {
            $business = $this->repo->create($dto->toArray());
            event(new BusinessCreated($business));
            return $business;
        });
    }

    public function update(string $id, UpdateBusinessDTO $dto): Business
    {
        $business = $this->findOrFail($id);
        return $this->repo->update($business->id, $dto->toArray());
    }

    public function uploadLogo(string $id, \Illuminate\Http\UploadedFile $file): Business
    {
        $business = $this->findOrFail($id);

        // Hapus logo lama
        if ($business->logo) {
            Storage::disk('public')->delete($business->logo);
        }

        $path = $file->store("businesses/{$id}/logo", 'public');

        return $this->repo->update($id, ['logo' => $path]);
    }

    public function delete(string $id): void
    {
        $business = $this->findOrFail($id);
        $this->repo->delete($business->id);
    }
}
```

app/Modules/Tenant/Services/BranchService.php

```php
<?php

namespace App\Modules\Tenant\Services;

use App\Modules\Tenant\DTOs\CreateBranchDTO;
use App\Modules\Tenant\Models\Branch;
use App\Modules\Tenant\Repositories\Contracts\BranchRepositoryInterface;
use App\Shared\Exceptions\VeloraException;
use App\Shared\Services\BaseService;
use Illuminate\Pagination\LengthAwarePaginator;

class BranchService extends BaseService
{
    public function __construct(
        private readonly BranchRepositoryInterface $repo
    ) {}

    public function getByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
    {
        return $this->repo->findByBusiness($businessId, $filters);
    }

    public function findOrFail(string $id): Branch
    {
        $branch = $this->repo->findById($id);

        if (!$branch) {
            throw VeloraException::notFound('Branch');
        }

        return $branch;
    }

    public function create(CreateBranchDTO $dto, int $maxBranches = -1): Branch
    {
        // Cek limit subscription
        if ($maxBranches !== -1) {
            $count = $this->repo->countByBusiness($dto->businessId);
            if ($count >= $maxBranches) {
                throw new VeloraException(
                    "Batas maksimum cabang ({$maxBranches}) telah tercapai. Upgrade paket untuk
```

```
                403,
                'SUBSCRIPTION_LIMIT_REACHED'
            );
        }
    }

    $branch = $this->repo->create($dto->toArray());

    // Jika is_main = true, update branch lain
    if ($dto->isMain) {
        $this->repo->setMainBranch($branch->id, $dto->businessId);
    }

    return $branch->fresh();
}

public function update(string $id, array $data): Branch
{
    return $this->repo->update($id, $data);
}

public function setAsMain(string $branchId, string $businessId): Branch
{
    $branch = $this->findOrFail($branchId);

    if ($branch->business_id !== $businessId) {
        throw VeloraException::forbidden('BRANCH_ACCESS_DENIED');
    }

    $this->repo->setMainBranch($branchId, $businessId);

    return $branch->fresh();
}

public function delete(string $id): void
{
    $branch = $this->findOrFail($id);

    if ($branch->is_main) {
        throw new VeloraException('Tidak dapat menghapus cabang utama.', 422, 'CANNOT_DELETE
    }

    $this->repo->delete($branch->id);
```

```
    }
}
```

11. Controllers

app/Modules/Tenant/Controllers/BusinessController.php

```php
<?php

namespace App\Modules\Tenant\Controllers;

use App\Modules\Tenant\DTOs\CreateBusinessDTO;
use App\Modules\Tenant\DTOs\UpdateBusinessDTO;
use App\Modules\Tenant\Requests\StoreBusinessRequest;
use App\Modules\Tenant\Requests\UpdateBusinessRequest;
use App\Modules\Tenant\Resources\BusinessResource;
use App\Modules\Tenant\Services\BusinessService;
use App\Shared\Controllers\BaseController;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class BusinessController extends BaseController
{
    public function __construct(private readonly BusinessService $service) {}

    public function index(Request $request): JsonResponse
    {
        $paginator = $this->service->getAll($request->all());
        return $this->paginated($paginator, BusinessResource::collection($paginator));
    }

    public function store(StoreBusinessRequest $request): JsonResponse
    {
        $business = $this->service->create(
            CreateBusinessDTO::fromRequest($request->validated())
        );
        return $this->created(new BusinessResource($business));
    }

    public function show(string $id): JsonResponse
    {
        $business = $this->service->findOrFail($id);
        return $this->success(new BusinessResource($business));
    }
```

```php
    public function update(UpdateBusinessRequest $request, string $id): JsonResponse
    {
        $business = $this->service->update(
            $id,
            UpdateBusinessDTO::fromRequest($request->validated())
        );
        return $this->success(new BusinessResource($business), 'Business updated.');
    }

    public function uploadLogo(Request $request, string $id): JsonResponse
    {
        $request->validate([
            'logo' => ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
        ]);

        $business = $this->service->uploadLogo($id, $request->file('logo'));
        return $this->success(new BusinessResource($business), 'Logo updated.');
    }

    public function destroy(string $id): JsonResponse
    {
        $this->service->delete($id);
        return $this->noContent();
    }
}
```

app/Modules/Tenant/Controllers/BranchController.php

```php
<?php

namespace App\Modules\Tenant\Controllers;

use App\Modules\Tenant\DTOs\CreateBranchDTO;
use App\Modules\Tenant\Requests\StoreBranchRequest;
use App\Modules\Tenant\Requests\UpdateBranchRequest;
use App\Modules\Tenant\Resources\BranchResource;
use App\Modules\Tenant\Services\BranchService;
use App\Shared\Controllers\BaseController;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class BranchController extends BaseController
{
    public function __construct(private readonly BranchService $service) {}

    public function index(Request $request): JsonResponse
    {
        $businessId = auth()->user()->business_id;
        $paginator  = $this->service->getByBusiness($businessId, $request->all());
        return $this->paginated($paginator, BranchResource::collection($paginator));
    }

    public function store(StoreBranchRequest $request): JsonResponse
    {
        $businessId = auth()->user()->business_id;
        $branch = $this->service->create(
            CreateBranchDTO::fromRequest($request->validated(), $businessId)
        );
        return $this->created(new BranchResource($branch));
    }

    public function show(string $id): JsonResponse
    {
        $branch = $this->service->findOrFail($id);
        return $this->success(new BranchResource($branch));
    }

    public function update(UpdateBranchRequest $request, string $id): JsonResponse
    {
```

```
        $branch = $this->service->update($id, $request->validated());
        return $this->success(new BranchResource($branch), 'Branch updated.');
    }

    public function setMain(string $id): JsonResponse
    {
        $businessId = auth()->user()->business_id;
        $branch = $this->service->setAsMain($id, $businessId);
        return $this->success(new BranchResource($branch), 'Branch set as main.');
    }

    public function destroy(string $id): JsonResponse
    {
        $this->service->delete($id);
        return $this->noContent();
    }
}
```

12. Routes — `app/Modules/Tenant/Routes/api.php`

```php
<?php

use App\Modules\Tenant\Controllers\BusinessController;
use App\Modules\Tenant\Controllers\BranchController;
use Illuminate\Support\Facades\Route;

Route::middleware(['auth:sanctum', 'tenant.scope'])->group(function () {

    // Business
    Route::prefix('business')->name('business.')->group(function () {
        Route::get('/',           [BusinessController::class, 'show'])->name('show')
            ->defaults('id_from', 'auth'); // override in controller: show user's own business
        Route::put('/',           [BusinessController::class, 'update'])->name('update');
        Route::post('/logo',      [BusinessController::class, 'uploadLogo'])->name('logo');
    });

    // Branches
    Route::apiResource('branches', BranchController::class);
    Route::post('branches/{branch}/set-main', [BranchController::class, 'setMain'])
        ->name('branches.set-main');
});
```

13. Tenant Isolation Middleware (Update)

app/Shared/Middleware/EnsureTenantScope.php

```php
<?php

namespace App\Shared\Middleware;

use App\Modules\Tenant\Models\Business;
use App\Shared\Helpers\ApiResponse;
use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;

class EnsureTenantScope
{
    public function handle(Request $request, Closure $next)
    {
        $user = $request->user();

        if (!$user?->business_id) {
            return ApiResponse::forbidden('No business context found.', 'NO_TENANT');
        }

        // Cache business per-request (bukan per-session)
        $business = Cache::remember(
            "business:{$user->business_id}",
            now()->addMinutes(5),
            fn() => Business::find($user->business_id)
        );

        if (!$business || !$business->isActive()) {
            return ApiResponse::forbidden('Business is inactive or suspended.', 'BUSINESS_INACTI
        }

        // Share ke seluruh lifecycle request
        app()->instance('current.business',    $business);
        app()->instance('current.business_id', $business->id);

        return $next($request);
```

```
    }
}
```

14. Event

app/Modules/Tenant/Events/BusinessCreated.php

```php
<?php

namespace App\Modules\Tenant\Events;

use App\Modules\Tenant\Models\Business;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class BusinessCreated
{
    use Dispatchable, SerializesModels;

    public function __construct(public readonly Business $business) {}
}
```

Checklist Module Tenant

- [x] Migration: `businesses`, `branches`, alter `users`
- [x] Model: `Business` (UUID, slug auto-gen, enum cast, relations)
- [x] Model: `Branch` (UUID, enum cast, is_main logic)
- [x] Enums: `BusinessStatus`, `BranchStatus` dengan label Bahasa Indonesia
- [x] Repository Interface + Eloquent Implementation (Business & Branch)
- [x] RepositoryServiceProvider binding
- [x] DTOs: Create/Update Business, Create Branch
- [x] Form Requests: Store/Update Business, Store/Update Branch
- [x] Resources: BusinessResource, BranchResource
- [x] Services: BusinessService (upload logo), BranchService (limit check, set-main)

- [x] Controllers: BusinessController, BranchController
- [x] Routes: CRUD + set-main + upload-logo
- [x] EnsureTenantScope middleware (dengan Redis cache 5 menit)
- [x] Event: BusinessCreated

> **Next**: Module Subscription — Plans, billing, feature limits, trial, grace period